P0920R1
Precalculated hash values in lookup

Published Proposal, 2019-01-07

This version:
http://wg21.link/P0920r1

Author:
Mateusz Pusz (Epam Systems) mateusz.pusz@gmail.com

Audience:
LWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
github.com/mpusz/wg21_papers/blob/master/src/0920_precalculated_hash_values_in_lookup.bs

---

Abstract

This proposal extends the interface of unordered containers with the member function overloads that have one additional argument taking a precalculated hash value for the value being queried.

Table of Contents

§ 1. Motivation and Scope

In business scenarios it often happens that we have to look for the same keyword in more than one container at a time. Doing that is expensive right now as it forces hash value recalculation on every lookup.

With the changes proposed by this paper the following code will calculate the hash value only once per each run of the function `update()`:

```
std::array<std::unordered_map<std::string, int>, array_size> maps;

void update(const std::string& user)
{
  const auto hash = maps.front().hash_function()(user);
  for(auto& m : maps) {
    auto it = m.find(user, hash);
    // ...
  }
}
```

§ 2. Prior Work

Proposed feature was implemented in the tsl::hopscotch_map and proved to deliver significant performance improvements.

§ 3. Impact On The Standard

This proposal modifies the unordered associative containers in `<unordered_map>` and `<unordered_set>` by overloading the lookup member functions with member function templates having one additional parameter.

There are no language changes.

All existing C++17 code is unaffected.

§ 4. Considered Alternatives

§ 4.1. Stateful hash object

Similar, although a bit slower, behavior can be obtained with usage of a stateful hash object that introduces additional branch on every lookup:

```
template<typename Key, typename Hash>
struct hash_cache {
  inline static std::pair<Key, std::size_t> cache;
  size_t operator()(const Key& k) const
  {
    std::size_t val{};
    if (k != cache.first) {
      cache.first = k;
      cache.second = Hash()(k);
    }
    val = cache.second;
    return val;
  }
};
```

However, the case complicates in a multithreaded environment where synchronization has to be introduced to such a `hash_cache_sync` helper class:

```
template<typename Key, typename Hash>
struct hash_cache_sync {
  inline static std::mutex m;
  inline static std::pair<Key, std::size_t> cache;
  size_t operator()(const Key& k) const
  {
    std::size_t val{};
    {
      std::scoped_lock lock(m);
      if (k != cache.first) {
        cache.first = k;
        cache.second = Hash()(k);
      }
      val = cache.second;
    }
    return val;
  }
};
```

Such synchronization nearly negates all benefits of having a cache.

Another problem with that solution happens in the case of the heterogeneous lookup introduced by [p0919r3]:

```
struct string_hash {
  using transparent_key_equal = std::equal_to<>;
  std::pair<???, std::size_t> cache;
  std::size_t operator()(std::string_view txt) const;
  std::size_t operator()(const std::string& txt) const;
  std::size_t operator()(const char* txt) const;
};
```

In such a case there is no one good `Key` type to be used for storage in a cache. Additional conversions and object constructions will always be involved which negates all benefits of having the heterogeneous lookup feature.

§ 5. Proposed Wording

The proposed changes are relative to the working draft of the standard as of [n4791].

Modify **21.2.7 [unord.req]** paragraph 11 as follows:

Add new paragraph 11.23 in **21.2.7 [unord.req]**:

> - `hk` and `hke` denote precalculated hash values with `hf` for `k` and `ke`, respectively,

Modify table 70 in section **21.2.7 [unord.req]** as follows:

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`b.find(k)` `b.find(k, hk)`	`iterator`; `const_iterator` for const `b`.	Returns an iterator pointing to an element with key equivalent to `k`, or `b.end()` if no such element exists.	Average case O(1), worst case O(`b.size()`).
`a_tran.find(ke)` `a_tran.find(ke, hke)`	`iterator`; `const_iterator` for const `a_tran`.	Returns an iterator pointing to an element with key equivalent to `ke`, or `a_tran.end()` if no such element exists.	Average case O(1), worst case O(`a_tran.size()`).
`b.count(k)` `b.count(k, hk)`	`size_type`	Returns the number of elements with key equivalent to `k`.	Average case O(`b.count(k)`), worst case O(`b.size()`).
`a_tran.count(ke)` `a_tran.count(ke, hke)`	`size_type`	Returns the number of elements with key equivalent to `ke`.	Average case O(`a_tran.count(ke)`), worst case O(`a_tran.size()`).
`b.contains(k)`	bool	Equivalent to `b.find(k) != b.end()`	Average case O(1), worst case O(`b.size()`)
`b.contains(k, hk)`	bool	Equivalent to `b.find(k, hk) != b.end()`	Average case O(1), worst case O(`b.size()`)
`a_tran.contains(ke)`	bool	Equivalent to `a_tran.find(ke) != a_tran.end()`	Average case O(1), worst case O(`a_tran.size()`)
`a_tran.contains(ke, hke)`	bool	Equivalent to `a_tran.find(ke, hke) != a_tran.end()`	Average case O(1), worst case O(`a_tran.size()`)
`b.equal_range(k)` `b.equal_range(k, hk)`	`pair<iterator, iterator>`; `pair<const_iterator, const_iterator>` for const `b`.	Returns a range containing all elements with keys equivalent to `k`. Returns `make_pair(b.end(), b.end())` if no such elements exist.	Average case O(`b.count(k)`), worst case O(`b.size()`).
`a_tran.equal_range(ke)` `a_tran.equal_range(ke, hke)`	`pair<iterator, iterator>`; `pair<const_iterator, const_iterator>` for const `a_tran`.	Returns a range containing all elements with keys equivalent to `ke`. Returns `make_pair(a_tran.end(), a_tran.end())` if no such elements exist.	Average case O(`a_tran.count(ke)`), worst case O(`a_tran.size()`).

Add a new paragraph (19) in **21.2.7 [unord.req]**:

> Precalculated `hash` value provided as a second argument to lookup member functions (`find`, `count`, `contains`, `equal_range`) shall be calculated using the `hasher` type of the container; no diagnostic required.

Add the following changes to:

- **21.5.4.1 [unord.map.overview]**
- **21.5.5.1 [unord.multimap.overview]**
- **21.5.6.1 [unord.set.overview]**
- **21.5.7.1 [unord.multiset.overview]**

```
iterator       find(const key_type& k);
const_iterator find(const key_type& k) const;
iterator       find(const key_type& k, size_t hash);
const_iterator find(const key_type& k, size_t hash) const;
template <class K> iterator       find(const K& k);
template <class K> const_iterator find(const K& k) const;
template <class K> iterator       find(const K& k, size_t hash);
template <class K> const_iterator find(const K& k, size_t hash) const;
size_type count(const key_type& k) const;
size_type count(const key_type& k, size_t hash) const;
template <class K> size_type count(const K& k) const;
template <class K> size_type count(const K& k, size_t hash) const;
bool contains(const key_type& k) const;
bool contains(const key_type& k, size_t hash) const;
template <class K> bool contains(const K& k) const;
template <class K> bool contains(const K& k, size_t hash) const;
pair<iterator, iterator>             equal_range(const key_type& k);
pair<const_iterator, const_iterator> equal_range(const key_type& k) const;
pair<iterator, iterator>             equal_range(const key_type& k, size_t hash);
pair<const_iterator, const_iterator> equal_range(const key_type& k, size_t hash) const;
template <class K> pair<iterator, iterator>             equal_range(const K& k);
template <class K> pair<const_iterator, const_iterator> equal_range(const K& k) const;
template <class K> pair<iterator, iterator>             equal_range(const K& k, size_t h
template <class K> pair<const_iterator, const_iterator> equal_range(const K& k, size_t h
```

§ 6. Feature Testing

Add the following row to a **Table 36** in **16.3.1 [support.limits.general]** paragraph 3:

Macro name	Value	Header(s)
...		
__cpp_lib_generic_associative_lookup	201304L	<map> <set>
__cpp_lib_generic_unordered_lookup	201811L	<unordered_map> <unordered_set>
__cpp_lib_generic_unordered_hash_lookup		<unordered_map> <unordered_set>

§ 7. Implementation Experience

Changes related to that proposal are partially implemented in GitHub repo against libc++ 7.0.0.

Simple performance tests provided there proved nearly:

- 20% performance gain for short text
- 50% performance gain for long text

§ 8. Revision History

§ 8.1. r0 ➡ r1 [diff]

- Rebased to [n4791]
- Simplified wording by aggregating rows in a table (where possible) and providing overview wording once for all the containers
- Feature test macro name changed

§ 9. Acknowledgements

Special thanks and recognition goes to Epam Systems for supporting my membership in the ISO C++ Committee and the production of this proposal.

§ References

§ Normative References

[N4791]
Richard Smith. Working Draft, Standard for Programming Language C++. 26 November 2018. URL: https://wg21.link/n4791

[P0919R3]
Mateusz Pusz. Heterogeneous lookup for unordered containers. 9 November 2018. URL: https://wg21.link/p0919r3